

Human-Actionable Policy Distillation through Variational Prototype Selection

Ethan Ouellette & Braeden Shephard

March 20, 2026

1 Project Goals

The primary objective of this project is to automate the selection of interpretability-enabling prototypes in the Prototype-Wrapper Network (PW-Net) framework. While Deep Reinforcement Learning (RL) has achieved state-of-the-art results in complex domains, these models remain opaque. PW-Net addresses this by wrapping a black-box encoder and forcing decisions to be based on human-friendly prototypes. However, the current model requires humans to manually define these prototypical states, which is not scalable in environments where expert concepts are difficult to define.

Our project investigates whether **action-space clustering** can automatically identify the minimal set of "cases" required for a PW-Net to maintain performance while remaining transparent. This is directly related to finding heuristically simpler policies in a given Rashomon set for a task.

Complete success is defined as:

- Implementing an automated selection algorithm that clusters actions in the (steering, acceleration, braking) space to identify representative training states.
- Achieving an average reward in the Car Racing domain within 10% (?) of the original black-box agent while using fewer than 10 (?) prototypes.

Partial success would involve demonstrating the performance-interpretability trade-off—identifying the "breaking point" where reducing the number of prototypes causes the wrapper to lose the ability to approximate the black-box oracle.

2 Technical Approach

We will modify an existing algorithm focused on the OpenAI Gym Car Racing domain.

2.1 Data Generation and Action-Space Clustering

We will first collect a dataset $\mathcal{D} = \{(s, \pi_{bb}(s))\}_{i=0}^n$ of state-action pairs by running a pre-trained PPO black-box agent. Since the action space $A \in \mathbb{R}^3$ corresponds to steering, braking, and acceleration, we will apply clustering to these action tuples. For each of the k resulting clusters, we will select the state s closest to the cluster centroid to serve as the automated prototype $p_{i,j}$. We will also experiment with sampling of prototypes from clusters based on distance to centroid to reduce overfitting.

2.2 Wrapper Integration and Optimization

The automated prototypes will be encoded through the original agent's encoder f_{enc} and used as latent prototypes $p_{i,j}$. We will train the projection networks $h_{i,j}$ using gradient descent to minimize

the loss between the wrapper output a'_i and the black-box action a . The final action will be calculated as a weighted sum of similarity scores:

$$a'_i = \sum_{j=1}^{N_i} W'_{i,j} \text{sim}(z_{i,j}, p_{i,j}) \quad (1)$$

where sim is the difference-of-logarithms similarity function. We will then perform a sweep of $k \in \{2, 4, 8, 16, 32, 64\}$ to analyze how cluster density affects cumulative reward. Intuitively, wider sweeping of this cluster space will lead to greater number of prototypes leading to a more complex/less intuitive model. To quantify the benefits of a lighter, more intuitive model, we can also tune a λ parameter penalizing models for large prototype spaces and rewarding sparser, more easily human-actionable prototype spaces.

3 Logistics

The project scope is commensurate with our team size of 2. We will utilize the existing open-source code for PW-Net to avoid manual boilerplate implementation.

Resources and Sanity Check: Training individual wrappers is computationally efficient, typically requiring less than 30 minutes on a standard CPU. To ensure statistical significance, we will perform evaluations across 5 random seeds for each configuration. Accounting for debugging and parameter tuning, we estimate a total of 50 experiment runs, which is realistic within our three-week timeline.

Timeline:

- **Week 1:** Environment setup, black-box data collection, and clustering logic development.
- **Week 2:** Wrapper training sweeps across different cluster sizes (k).
- **Week 3:** Comparative analysis of MSE/Reward and final write-up.

References

- [1] Bontempelli et al. (2023). Concept-level Debugging of Part-Prototype Networks. *ICLR*.
- [2] Chen et al. (2019). This Looks Like That: Deep Learning for Interpretable Image Recognition. *NeurIPS*.
- [3] Kenny et al. (2023). Towards Interpretable Deep Reinforcement Learning with Human-Friendly Prototypes. *ICLR*.
- [4] Paes et al. (2023). On the inevitability of the Rashomon effect. *ISIT*.